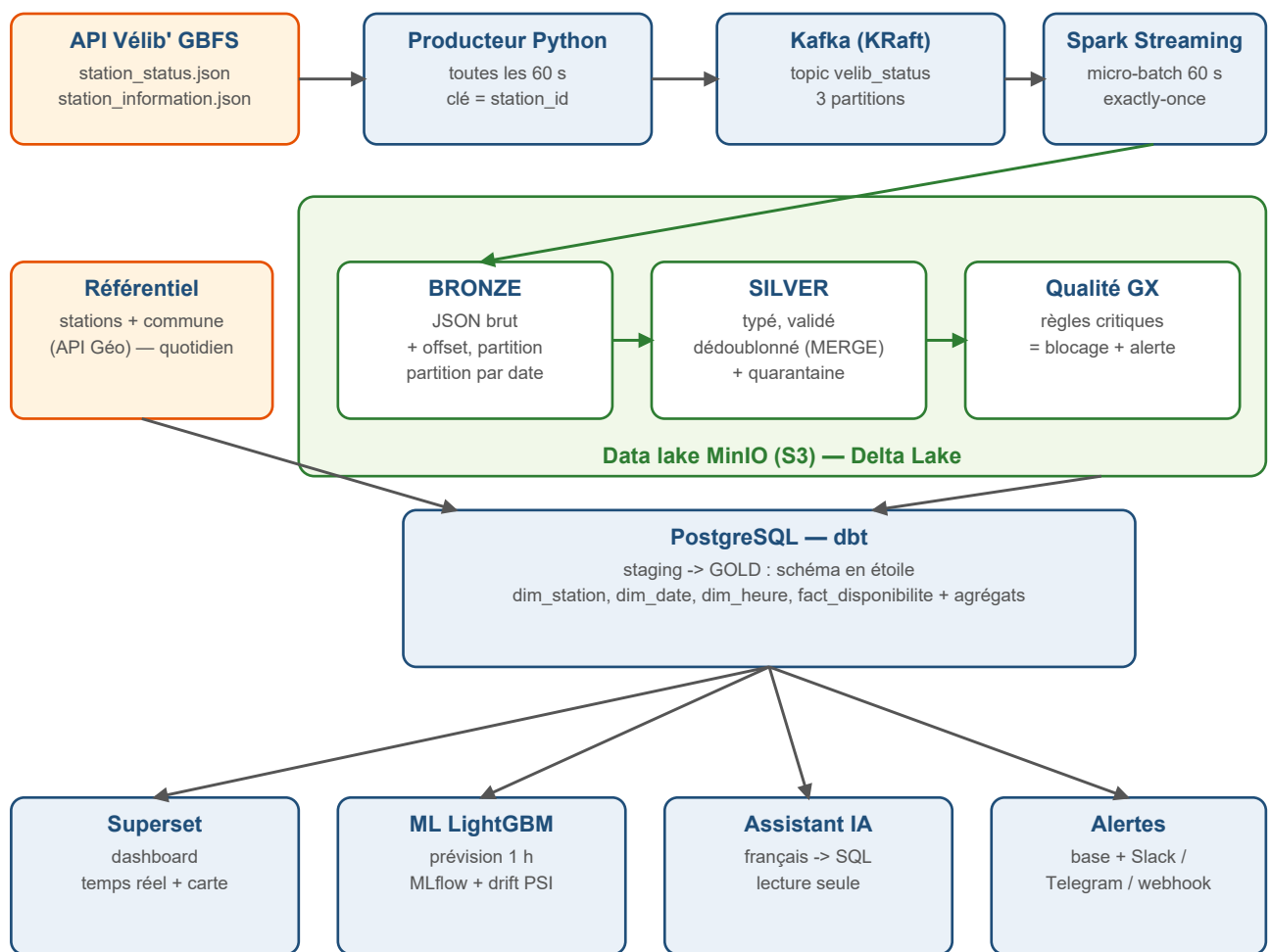


Vélib' Data Platform

Plateforme de données temps réel avec prévision ML, BI et assistant IA



Orchestration Airflow 3 • Monitoring Prometheus + Grafana

Document d'explication détaillée — état au 5 octobre 2026
Projet basé sur hoangsonww/End-to-End-Data-Pipeline (licence MIT)

Sommaire

1. Le projet en résumé
2. Les données Vélib'
3. Architecture d'ensemble
4. Ingestion temps réel : producteur et Kafka
5. Architecture médaillon avec Spark et Delta Lake
6. Qualité des données avec Great Expectations
7. Data warehouse avec dbt
8. Orchestration avec Airflow 3
9. Prévion ML, MLflow et drift
10. BI avec Superset
11. Assistant IA text-to-SQL
12. Monitoring et alertes
13. Déploiement : Docker, Kubernetes, AWS
14. CI/CD
15. Choix techniques et justifications
16. État d'avancement et prochaines étapes
17. Lancer le projet
18. Préparer l'entretien

1. Le projet en résumé

Le problème. Vélib' Métropole compte environ 1 500 stations. Pour l'utilisateur, trouver une station vide quand on veut un vélo, ou pleine quand on veut le rendre, est la principale frustration. Pour l'opérateur, anticiper ces situations permet de mieux planifier la régulation (camions qui redistribuent les vélos).

La solution. Une plateforme de données complète qui collecte l'état réel de toutes les stations chaque minute, le fiabilise en trois couches (bronze, silver, gold), le modélise en schéma en étoile, prédit la disponibilité à une heure et rend les résultats accessibles à tous : dashboard BI pour les analystes, assistant en langage naturel pour les non-techniciens.

L'histoire à raconter : « Je collecte des données réelles, je les fiabilise, je les modélise, je prédis, et je les rends accessibles à tout le monde. »

Les six objectifs du projet

Objectif	Comment	Outils
Collecter	Streaming de l'état des stations toutes les 60 s + référentiel quotidien	Python, Kafka, Spark Structured Streaming
Fiabiliser	Couches bronze / silver / gold, quarantaine, contrôles qualité bloquants	Delta Lake, MinIO, Great Expectations
Modéliser	Schéma en étoile, agrégats métiers, tests et documentation	dbt, PostgreSQL
Prédire	Vélos disponibles à 1 h par station, suivi du drift	LightGBM, Optuna, MLflow, PSI
Exposer	Dashboard et questions en français traduites en SQL	Superset, LLM (Ollama / Claude)
Industrialiser	Conteneurs, orchestration, monitoring, alertes, IaC, CI/CD	Docker, Airflow, Prometheus, Grafana, Helm, Terraform, GitHub Actions

Point de départ

Le projet part du dépôt open source [hoangsonww/End-to-End-Data-Pipeline](https://github.com/hoangsonww/End-to-End-Data-Pipeline) (licence MIT), une démonstration générique sur des données simulées (capteurs IoT et commandes). Le fichier LICENSE d'origine est conservé et le projet source sera crédité dans le README. Toute la partie métier a été réécrite : données réelles, médaillon, dbt, ML, assistant, alertes. L'infrastructure (Docker, Helm, Terraform, CI) est adaptée.

2. Les données Vélib'

La source est le flux open data **GBFS** (General Bikeshare Feed Specification) de Vélib' Métropole : gratuit, sans clé d'API, rafraîchi en continu. Deux fichiers sont utilisés :

Flux	Contenu	Fréquence d'appel
station_status.json	Pour chaque station : vélos disponibles (détail mécaniques / électriques), bornes libres, station installée / en location / en retour, horodatage last_reported	Toutes les 60 secondes
station_information.json	Nom, coordonnées GPS, capacité, code station	Une fois par jour

Exemple de relevé brut (simplifié) :

```
{"station_id": 213688169, "stationCode": "16107",  
  "num_bikes_available": 0,  
  "num_bikes_available_types": [{"mechanical": 0}, {"ebike": 0}],  
  "num_docks_available": 35, "is_installed": 1, "is_renting": 1,  
  "is_returning": 1, "last_reported": 1791220687}
```

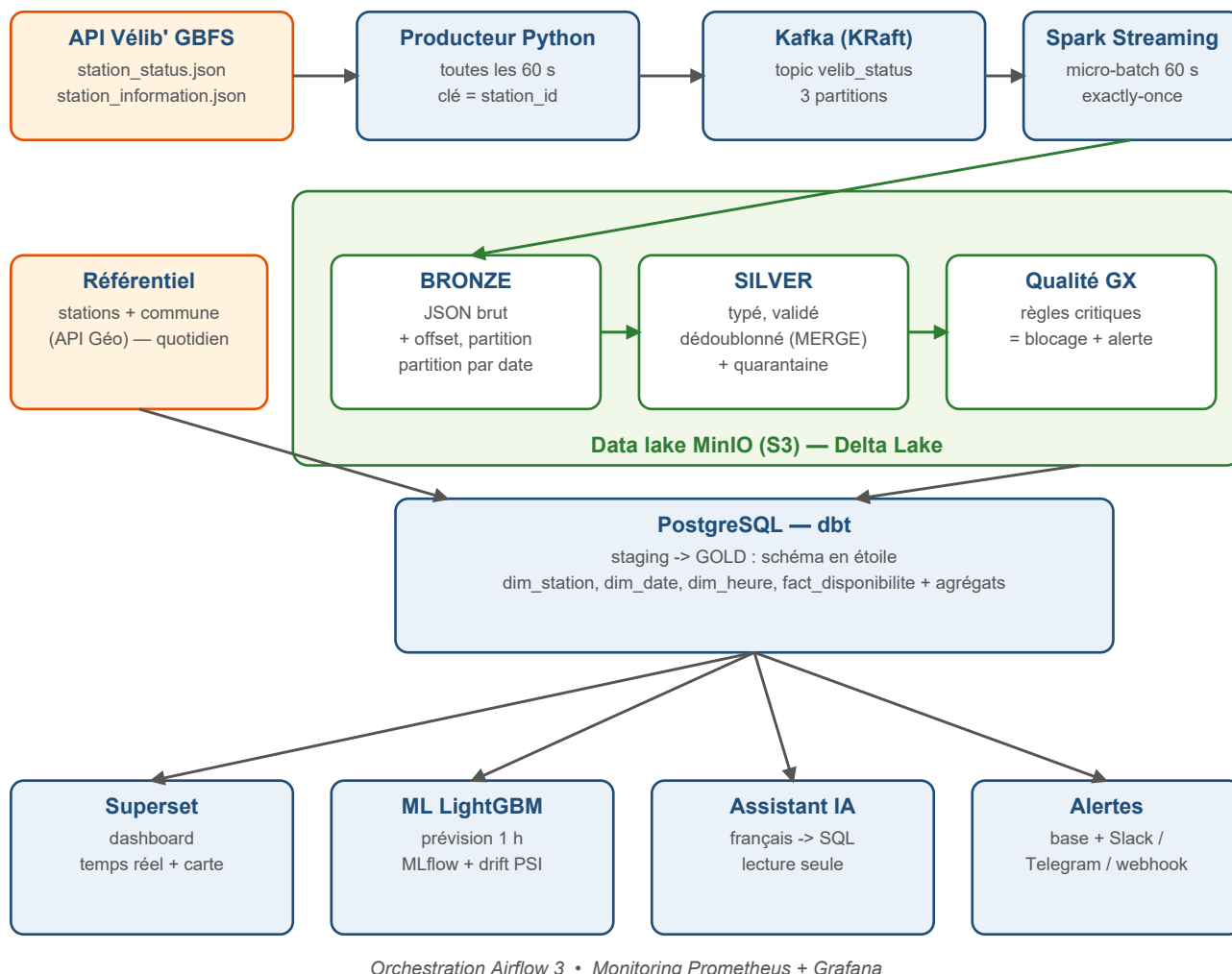
Enrichissement géographique

Le flux ne donne ni arrondissement ni commune. Le DAG du référentiel interroge l'**API Géo de l'État** (geo.api.gouv.fr) en géocodage inverse : pour Paris on obtient l'arrondissement (ex. « Paris 16e Arrondissement », code INSEE 75116), ailleurs la commune (ex. Boulogne-Billancourt, 92012). Seules les nouvelles stations sont géocodées, le résultat est conservé en base.

Volumétrie

- 1 519 stations dans le flux.
- Le producteur ne republie une station que si son *last_reported* a changé : environ 140 messages par minute au lieu de 1 519, soit environ 200 000 relevés par jour (≈ 10 fois moins de volume, sans perte d'information).
- Le référentiel de jours fériés français (2025-2028) est fourni à dbt sous forme de seed CSV.

3. Architecture d'ensemble



Le flux de données suit cinq étapes :

- **Ingestion** — un producteur Python interroge l'API et publie dans Kafka ; Spark Structured Streaming lit Kafka et écrit la couche **bronze** (brute) en Delta Lake sur MinIO.
- **Fiabilisation** — toutes les 15 minutes, un job Spark produit la couche **silver** (typée, validée, dédoublonnée) ; Great Expectations contrôle le lot avant chargement dans PostgreSQL.
- **Modélisation** — chaque heure, dbt construit la couche **gold** : schéma en étoile et agrégats.
- **Prédiction** — après chaque mise à jour du gold, le modèle champion prédit la disponibilité à 1 h ; le drift est mesuré chaque jour.
- **Exposition** — Superset, l'assistant IA et Grafana lisent le gold et les tables de monitoring.

Services de la stack Docker Compose

Service	Rôle	Port local
postgres	Warehouse velib_dw + bases airflow, mlflow, superset	5432
minio (+ minio-init)	Data lake S3 : buckets lake et mlflow	9000 / 9001 (console)
kafka	Broker Kafka 4 en mode KRaft (sans ZooKeeper)	9092
velib-producer	Collecte GBFS -> Kafka, métriques Prometheus	8000

Service	Rôle	Port local
spark-streaming	Job bronze Kafka -> Delta	4040 (UI), 8001 (métriques)
airflow-apiserver / scheduler / dag-processor	Orchestration (Airflow 3, LocalExecutor)	8080
mlflow	Suivi d'expériences et Model Registry	5001
statsd-exporter, prometheus, grafana	Monitoring technique	9102, 9090, 3000
superset (à venir)	Dashboard BI	8088
assistant (à venir)	Application text-to-SQL (Streamlit)	8501
spark-master / spark-worker (optionnel)	Cluster Spark standalone, profil spark-cluster	7077, 8081

La stack est dimensionnée pour environ 7 Go de RAM : Kafka sans ZooKeeper, streaming Spark en mode local (cluster optionnel), limites mémoire sur chaque conteneur, parallélisme Airflow limité à 4 tâches.

4. Ingestion temps réel : producteur et Kafka

Le producteur (kafka/producer.py)

Le code sépare les **fonctions pures** (validation du flux, construction des messages, sérialisation), testables sans réseau ni Kafka, des **entrées/sorties** (HTTP, Kafka).

- **Robustesse réseau** : session HTTP avec 5 tentatives et attente exponentielle sur erreurs réseau, 429 et 5xx, timeouts de connexion (5 s) et de lecture (20 s). Un flux invalide est journalisé sans arrêter le service.
- **Message brut** : l'objet station est publié tel quel, enveloppé de métadonnées (feed_last_updated, fetched_at, source). Aucune transformation métier : c'est le rôle de silver.
- **Clé Kafka = station_id** : tous les relevés d'une station vont dans la même partition, leur ordre est garanti.
- **Producteur idempotent** : acks=all + enable.idempotence, aucune perte ni doublon côté broker en cas de retry.
- **Capture de changement** : publication uniquement si last_reported a changé (option VELIB_ONLY_CHANGED).
- **Observabilité** : métriques Prometheus (messages publiés, erreurs de livraison, erreurs d'appel, durée, horodatage du dernier succès) ; arrêt propre sur SIGTERM.

Kafka

Kafka 4.1 en mode **KRaft** : le broker gère lui-même ses métadonnées, ZooKeeper (supprimé dans Kafka 4) n'est plus nécessaire. Le topic *velib_status* a 3 partitions et 3 jours de rétention. La création automatique de topics est désactivée : c'est le producteur qui crée le topic avec la bonne configuration.

Pourquoi Kafka plutôt qu'une écriture directe en base ? Kafka découple la collecte du traitement : si Spark ou la base tombent, les messages restent dans Kafka (3 jours) et sont rejoués au redémarrage. On peut aussi brancher d'autres consommateurs (alerting temps réel, autre application) sans toucher au producteur.

5. Architecture médaillon avec Spark et Delta Lake

Pourquoi trois couches ?

Couche	Contenu	Intérêt
Bronze	JSON brut + métadonnées Kafka (topic, partition, offset, timestamps), partitionné par date	Rien n'est perdu : on peut tout recalculer en cas de bug en aval
Silver	Données typées, validées, dédoublonnées ; vélos mécaniques / électriques séparés	Source fiable et réutilisable
Gold	Schéma en étoile et agrégats métiers dans PostgreSQL (dbt)	Prêt pour la BI, le ML et l'IA

Bronze : spark/bronze_stream_job.py

- Spark Structured Streaming lit Kafka par micro-batches de 60 secondes et écrit en Delta Lake dans `s3a://lake/bronze/velib_status`, partitionné par date Kafka (déterministe en cas de rejou).
- **Exactly-once** : le checkpoint mémorise les offsets traités et le sink Delta est idempotent (chaque micro-batch est une transaction identifiée par son batchId). Après un crash, Spark reprend au dernier offset validé, sans doublon.

- Un *StreamingQueryListener* expose à Prometheus le lag Kafka (offsets non lus), le débit et la durée des micro-batches — Structured Streaming ne passe pas par un consumer group, ce qui rend les outils de lag classiques aveugles.

Silver : spark/silver_job.py

- Lecture de la fenêtre bronze du run Airflow (+ 30 minutes de marge pour les messages tardifs), avec élagage par partition.
- Parsing avec un schéma explicite, conversion des timestamps en UTC, calcul mécaniques / électriques via une fonction d'ordre supérieur Spark SQL (aggregate) — pas d'UDF Python, donc pas de coût de sérialisation.
- Règles de validité : station_id présent, last_reported plausible (ni par défaut, ni dans le futur), vélos et bornes ≥ 0 , station connue du référentiel. Les lignes invalides partent en **quarantaine** (table Delta dédiée) avec leur motif de rejet.
- Dédoublonnage sur (station_id, last_reported) en gardant le message Kafka le plus récent.
- **MERGE** Delta idempotent : rejouer une fenêtre n'insère jamais deux fois le même relevé.
- Mode maintenance : OPTIMIZE (compaction des petits fichiers créés chaque minute) et VACUUM (7 jours).

MinIO joue le rôle de S3 en local ; le même code fonctionne sur Amazon S3 en changeant l'endpoint.

6. Qualité des données avec Great Expectations

Le lot silver est écrit dans une table tampon PostgreSQL (staging.velib_status_incoming), puis validé par une suite Great Expectations 1.x versionnée en JSON (great_expectations/suites/silver_velib_status.json).

Sévérité	Règles	Effet
Critique	Lot non vide ; station_id et last_reported non nuls ; couple (station_id, last_reported) unique ; vélos et bornes entre 0 et 150 ; mécaniques et électriques ≥ 0	La tâche Airflow échoue, le lot n'est PAS chargé, alerte data_quality
Avertissement	mécaniques + électriques = total (99 %) ; code station présent ; relevé le plus récent < 3 h ; délai d'ingestion < 30 min (95 %)	Journalisé et visible dans Grafana

Chaque résultat est historisé dans monitoring.data_quality_results. Si le lot est conforme, il est fusionné dans staging.velib_status avec INSERT ... ON CONFLICT DO NOTHING (idempotent).

7. Data warehouse avec dbt

Le projet dbt (dbt/velib) transforme le staging en couche gold. Les noms de colonnes sont en français : ils parlent aux utilisateurs du dashboard et aident le LLM de l'assistant à comprendre les questions.



Modèles

Modèle	Type	Description
stg_velib_status, stg_velib_stations	vue	Renommage, heure locale de Paris, zone (« Paris 16e » ou commune), département
dim_station	table	Une ligne par station : nom, zone, commune, arrondissement, capacité, coordonnées
dim_date	table	Calendrier 2025 -> +1 an : jours en français, week-end, jours fériés (seed)
dim_heure	table	0-23, tranche horaire, heure de pointe
fact_disponibilite	incrémental	Grain le plus fin : un relevé par station ; taux de remplissage, vide, pleine
fct_station_horaire	incrémental	Grille station × heure : état à la fin de chaque heure (forward fill) — base des séries temporelles et du ML
agg_remplissage_horaire	table	Profil horaire moyen par station, semaine vs week-end/férié
agg_stations_critiques	table	Temps passé vide / pleine, part du temps, rangs (top 10)
etat_actuel_stations	table	Dernier état de chaque station + statut, pour la carte
agg_disponibilite_zone_horaire	table	Comparaison par arrondissement / commune et par heure
ml_predictions_vs_reel	table	Prévisions confrontées aux valeurs observées 1 h plus tard

Points techniques à savoir expliquer

- **Incrémental** (stratégie delete+insert) : seuls les relevés chargés depuis le dernier run sont traités ; une fenêtre de 3 h est recalculée pour absorber les données tardives.
- **Temps vide / pleine** : chaque relevé est valable jusqu'au suivant, plafonné à 120 minutes pour qu'une station muette ne soit pas comptée vide pendant des jours.

- **Tests** : not_null, unique, relationships (intégrité de l'étoile), accepted_values, et deux tests génériques écrits à la main (accepted_range pour un taux entre 0 et 1, unicité composée) — sans dépendance externe.
- **Fraîcheur des sources** : avertissement après 30 min sans données, erreur après 2 h.
- **Droits** : la configuration grants donne SELECT sur gold aux rôles en lecture seule (Superset et assistant).
- **Documentation** : chaque colonne est décrite ; dbt docs generate produit le graphe de lignage, avec des *exposures* pour le dashboard, le modèle ML et l'assistant.

Qu'apporte dbt par rapport à du SQL dans Airflow ? Dépendances calculées automatiquement (ref), tests déclaratifs, documentation et lignage générés, modèles incrémentaux, environnements (dev / CI) — le SQL devient du code versionné, testé et revu comme du logiciel.

8. Orchestration avec Airflow 3

Airflow 3.3 (api-server, scheduler, dag-processor, LocalExecutor). Trois environnements Python isolés dans l'image évitent les conflits de dépendances : celui d'Airflow, un venv *pipeline* (PySpark, Delta, Great Expectations, LightGBM, MLflow) et un venv *dbt*.

DAG	Planification	Rôle
velib_stations_reference_dag	quotidien 03:00	station_information -> MinIO bronze -> géocodage -> staging
velib_silver_quality_dag	toutes les 15 min	Bronze -> silver (Spark) -> Great Expectations -> staging
dbt_gold_dag	horaire (:05)	freshness -> seed -> run -> test -> docs
ml_prediction_dag	après chaque gold (Asset)	Prévision à 1 h avec le modèle champion
ml_training_dag	hebdomadaire + sur drift	Entraînement, enregistrement, promotion
drift_monitoring_dag	quotidien 06:00	PSI ; drift -> alerte + réentraînement
data_freshness_dag	toutes les 5 min	Alerte si aucune donnée Kafka depuis 10 min
lake_maintenance_dag	quotidien 02:30	OPTIMIZE + VACUUM Delta

- Les **Assets** (ex-Datasets) relient producteurs et consommateurs : la prédiction se déclenche dès que le gold est mis à jour.
- Retries avec backoff exponentiel et callback d'échec qui crée une alerte.
- Code de sortie 99 = « rien à faire » (pas encore assez d'historique, pas de modèle) : la tâche est ignorée, pas en échec.

9. Pr vision ML, MLflow et drift

Objectif : pr dire le nombre de v los disponibles dans chaque station une heure plus tard.

Features (ml/features.py)

Famille	Features
Calendrier de l'heure cible	heure, jour de la semaine, week-end, jour f�ri� (connus � l'avance : pas de fuite)
�tat actuel	v�los � t, taux de remplissage, part d'�lectriques
Historique	v�los � t-1 h et t-2 h, moyenne 3 h, tendance 1 h, valeur � la m�me heure la veille et la semaine pr�c�dente
Station	capacit�, latitude, longitude, Paris ou non

Les retards sont calcul s par **jointure sur l'horodatage** et non par position de ligne : un trou dans la s rie donne une valeur manquante (que LightGBM g re nativement), jamais une valeur d cal e. Les stations muettes depuis plus de 24 h sont exclues.

Entra nement (ml/train.py)

- **Validation temporelle** : les heures les plus r centes (20 %) servent de test ; le pass  n'est jamais m lang  au futur.
- **R f rence na ve** : « dans une heure, comme maintenant » (persistance). Le mod le doit faire mieux, sinon il n'apporte rien — c'est la premi re chose qu'un jury demandera.
- **Optuna** : 15 essais sur une validation elle aussi temporelle (taux d'apprentissage, nombre de feuilles, r gularisation...), avec arr t pr coce. Objectif L1 = MAE.
- **MLflow** : param tres, MAE, RMSE, gain vs r f rence, importance des features, mod le avec signature, enregistrement dans le Model Registry.
- **Promotion** : alias « champion » si le nouveau mod le fait au moins aussi bien que le champion actuel * valu  sur le m me jeu de test*.

Pr diction et suivi en production

ml/predict.py charge models:/velib_bikes_1h@champion, pr dit pour la derni re heure compl te, borne le r sultat entre 0 et la capacit , et  crit ml.predictions_disponibilit . dbt compare ensuite pr dit et r el.

Drift : le PSI (ml/drift_psi.py)

Le **Population Stability Index** compare la distribution r cente d'une feature (24 h)   celle de l'entra nement, d coup e en d ciles :

$$PSI = \sum (part_actuelle - part_r f rence) \times \ln(part_actuelle / part_r f rence)$$

Lecture usuelle : moins de 0,1 stable ; 0,1   0,2   surveiller ; plus de 0,2 changement significatif. Au-del  de 0,2, le DAG  met une alerte et d clenche le r entra nement. Exemples de causes : gr ve, travaux, nouvelles stations, ajout massif de v los  lectriques.

10. BI avec Superset

Dashboard « V lib' en temps r el », connect  au gold avec un utilisateur en lecture seule :

- carte des stations color e par taux de remplissage (etat_actuel_stations) ;
- top 10 des stations le plus souvent vides et pleines (agg_stations_critiques) ;

- évolution horaire des vélos disponibles ; mécaniques vs électriques (agg_disponibilite_zone_horaire) ;
- comparaison par arrondissement / commune ; prévisions vs réalité (ml_predictions_vs_reel).

Le dashboard sera versionné dans bi_dashboards/ (export Superset) pour être recréé automatiquement.

11. Assistant IA text-to-SQL

Une application Streamlit : l'utilisateur pose une question en français, par exemple « Quelles stations sont le plus souvent vides le lundi matin ? ». Le LLM reçoit le schéma gold et les descriptions des colonnes issues de la documentation dbt, puis génère une requête SQL ; la réponse est affichée en texte, tableau et graphique.

Sécurité en profondeur

- Rôle PostgreSQL **assistant_ro** : SELECT sur gold uniquement, transactions en lecture seule par défaut, timeout de 10 secondes (déjà créé dans scripts/postgres).
- Analyse de la requête générée avant exécution : une seule instruction, uniquement SELECT, uniquement le schéma gold, fonctions dangereuses interdites.
- LIMIT imposé sur le résultat.

LLM : Ollama en local (gratuit, llama3.2:3b déjà installé) ou Claude via l'API Anthropic. Un jeu de 15 à 20 questions avec résultats attendus mesure le taux de bonnes réponses, qui sera publié dans le README.

12. Monitoring et alertes

Métriques techniques (Prometheus + Grafana)

Source	Métriques
Producteur	messages publiés, erreurs HTTP / livraison, durée d'appel, dernier succès
Spark streaming	lag Kafka, débit d'entrée et de traitement, durée des micro-batches
Airflow (StatsD -> statsd-exporter)	durée des DAGs et des tâches, échecs, tâches en file
MinIO	métriques du stockage
PostgreSQL (Grafana)	résultats qualité, alertes, PSI

Alertes (airflow/dags/common/alerting.py)

Chaque alerte est enregistrée dans monitoring.alerts (visible dans Grafana même sans canal externe), puis envoyée aux canaux configurés : webhook générique (n8n), Slack, Telegram. Un anti-spam empêche de renvoyer la même alerte plusieurs fois dans la même fenêtre.

Type	Déclencheur
dag_failure	Échec définitif d'une tâche (après retries)
data_quality	Règle critique Great Expectations violée
drift	PSI > 0,2 sur au moins une feature
freshness	Aucun message Kafka depuis 10 minutes

13. Déploiement : Docker, Kubernetes, AWS

- **Docker Compose** : toute la plateforme en local, une commande (make up).
- **Kubernetes** : chart Helm (helm/e2e-pipeline) adapté aux services gardés et ajoutés, déployé sur un cluster local (Minikube / kind) avec values-onprem.yaml.
- **AWS avec Terraform** : EKS, RDS PostgreSQL, réseau. Version réduite pour limiter le coût : une seule zone de nœuds, petites instances, RDS non multi-AZ, NAT Gateway évitée si possible, alerte de budget (10 €). **terraform apply uniquement après validation manuelle du plan**, et terraform destroy le jour même.

14. CI/CD

Le workflow GitHub Actions sera adapté pour exécuter à chaque push :

- lint (flake8, black, isort) et tests Python (pytest) ;
- dbt compile et dbt test sur une base PostgreSQL de test (service du workflow) ;
- build des images Docker ; validation de docker-compose ;
- terraform fmt -check et terraform validate ; helm lint.

15. Choix techniques et justifications

Choix	Pourquoi
Kafka plutôt qu'une écriture directe	Découplage, tampon de 3 jours, rejeu, plusieurs consommateurs possibles
Kafka KRaft sans ZooKeeper	ZooKeeper est supprimé dans Kafka 4 ; un service et ~300 Mo de RAM en moins
Delta Lake	Transactions ACID sur un stockage objet, MERGE, time travel, OPTIMIZE / VACUUM, schéma imposé
Médaille	Séparer brut / fiable / métier : traçabilité, recalcul possible, responsabilités claires
Spark local pour le streaming	Le volume (~140 messages/min) ne justifie pas un cluster ; le cluster reste disponible en option
dbt	SQL testé, documenté, versionné, avec lignage et incrémental
PostgreSQL comme warehouse	Gratuit, suffisant pour ce volume ; même logique dbt transposable à BigQuery / Snowflake
LightGBM	Excellent sur données tabulaires, gère les valeurs manquantes, rapide, léger (vs XGBoost ~300 Mo)
Airflow 3	Airflow 2 est en fin de vie depuis avril 2026 ; Assets pour la planification orientée données
Venvs séparés	dbt, Great Expectations, MLflow et Airflow ont des dépendances incompatibles entre elles
MinIO (build pgsty/minio)	API S3 compatible ; les images officielles MinIO ne sont plus publiées sur Docker Hub
Ollama par défaut	LLM local et gratuit ; Claude en option pour une meilleure précision

Et avec 100 fois plus de données ?

- Kafka : plus de partitions et plusieurs brokers (réplication 3).
- Spark : passage en cluster (Kubernetes ou EMR/Databricks), le code ne change pas.
- Stockage : Amazon S3 à la place de MinIO ; Delta reste identique (LogStore DynamoDB si plusieurs écrivains).
- Warehouse : Snowflake, BigQuery ou Redshift ; les modèles dbt sont portables.
- Airflow : CeleryExecutor ou KubernetesExecutor ; le job silver en KubernetesPodOperator.

16. État d'avancement et prochaines étapes

Blocage actuel : le disque C: est plein (environ 0,2 Go libre ; l'image disque de Docker occupe 56 Go). Docker s'est arrêté pendant la construction des images. Il faut libérer au moins 25 Go pour continuer.

Phase	Statut	Détail
0 — Mise en place	Fait	Dépôt importé, historique Git propre, licence MIT conservée
1 — Simplification	Fait	MySQL, MongoDB, Redis, InfluxDB, Elasticsearch, API .NET, Snowflake supprimés
2 — Données Vélib'	Fait et testé	Producteur en fonctionnement : 1 519 stations publiées dans Kafka
3 — Médaille	Bronze testé	Bronze Delta écrit dans MinIO, lag 0 ; silver + GX écrits, à exécuter
4 — dbt	Écrit	Modèles, tests, docs et DAG écrits ; à exécuter dès que l'image Airflow est construite
5 — Superset	À faire	Service, datasets, dashboard, export
6 — ML	Écrit	Features, entraînement, prédiction, PSI, DAGs ; MLflow à démarrer

Phase	Statut	Détail
7 — Assistant IA	À faire	Application, garde-fous SQL, jeu d'évaluation
8 — Monitoring	Partiel	Alertes, fraîcheur, métriques et statsd écrits ; Prometheus/Grafana à configurer
9 — Kubernetes	À faire	Adapter le chart Helm, déployer en local
10 — AWS	À faire	Adapter Terraform ; apply seulement avec ton accord
11 — CI/CD	À faire	Adapter le workflow GitHub Actions
12 — README	À faire	README FR/EN, captures, schéma, résultats

17. Lancer le projet

```

cp .env.example .env      # configuration locale
make build                 # construit les images (Airflow, Spark, producteur, MLflow)
make up                   # démarre la stack
make urls                 # liste les interfaces
make test                 # tests unitaires

```

Interface	Adresse
Airflow	http://localhost:8080
Console MinIO (data lake)	http://localhost:9001
MLflow	http://localhost:5001
Spark UI (streaming)	http://localhost:4040
Grafana	http://localhost:3000
Superset / Assistant	http://localhost:8088 / http://localhost:8501

18. Préparer l'entretien

Pourquoi Kafka plutôt que d'écrire directement dans la base ?

Kafka découple production et consommation. Si Spark ou la base tombent, les messages restent 3 jours dans Kafka et sont rejoués. On peut ajouter des consommateurs sans toucher au producteur, et absorber des pics.

Que se passe-t-il si le job Spark Streaming plante ? Comment évites-tu les doublons ?

Le checkpoint conserve les offsets du dernier micro-batch validé ; au redémarrage Spark reprend là. Le sink Delta est idempotent (transaction par batchId), donc pas de doublon en bronze. En silver, le MERGE sur (station_id, last_reported) et l'ON CONFLICT DO NOTHING en staging garantissent l'idempotence de bout en bout.

Pourquoi trois couches bronze / silver / gold ?

Bronze garde le brut pour pouvoir tout recalculer ; silver est la vérité nettoyée et réutilisable ; gold est modélisé pour un usage métier. Un bug de transformation se corrige en rejouant depuis bronze.

Qu'apporte dbt par rapport à du SQL dans Airflow ?

Les dépendances (ref), les tests, la documentation et le lignage, l'incrémental, la portabilité entre warehouses : le SQL devient du code testé et versionné.

Comment évites-tu la fuite de données dans ton modèle ?

Split strictement temporel, features calculées uniquement avec le passé (lags par jointure sur l'horodatage), variables calendaires de l'heure cible seulement (connues à l'avance), comparaison des modèles sur le même test.

Qu'est-ce que le PSI et pourquoi le seuil de 0,2 ?

Mesure de l'écart entre deux distributions découpées en classes. 0,2 est un seuil empirique issu du scoring de crédit : au-delà, la population a changé de façon significative et le modèle risque de se dégrader.

Comment sécurises-tu l'assistant text-to-SQL ?

Défense en profondeur : rôle PostgreSQL en lecture seule limité à gold avec timeout, vérification que la requête est un unique SELECT sur gold, LIMIT imposé. Même si le LLM est manipulé, la base refuse toute écriture.

Combien coûte ta plateforme sur AWS et comment la réduire ?

Le coût réel sera mesuré et noté dans le README. Leviers : une seule zone, instances spot, RDS non multi-AZ, pas de NAT Gateway, arrêt hors démonstration (terraform destroy), budget avec alerte.

Que changerais-tu si le volume était multiplié par 100 ?

Partitions et brokers Kafka, Spark en cluster, S3, warehouse cloud (Snowflake/BigQuery), exécuteur Kubernetes pour Airflow — l'architecture ne change pas, seuls les composants passent à l'échelle.

Pour le CV

Vélib' Data Platform — Plateforme de données temps réel avec prévision ML. Pipeline batch et streaming sur les données réelles Vélib' (Kafka, Spark, Delta Lake) avec architecture médaillon, data warehouse dbt et dashboard Superset. Prévision de disponibilité par station (LightGBM, MLflow) avec détection de drift et assistant IA text-to-SQL. Orchestré par Airflow, déployé sur Kubernetes et AWS (Terraform, Helm), CI/CD GitHub Actions.